Relatório — Parte 2: Transmissão para a nuvem e visualização (Fog/Cloud Computing)

CardioIA — Fase 3 · Victor Copque dos Reis (RM566821) · Victor Hugo Ferreira Rolim (RM568006)

Projeto no Wokwi: https://wokwi.com/projects/479418030587973633

1. Visão geral

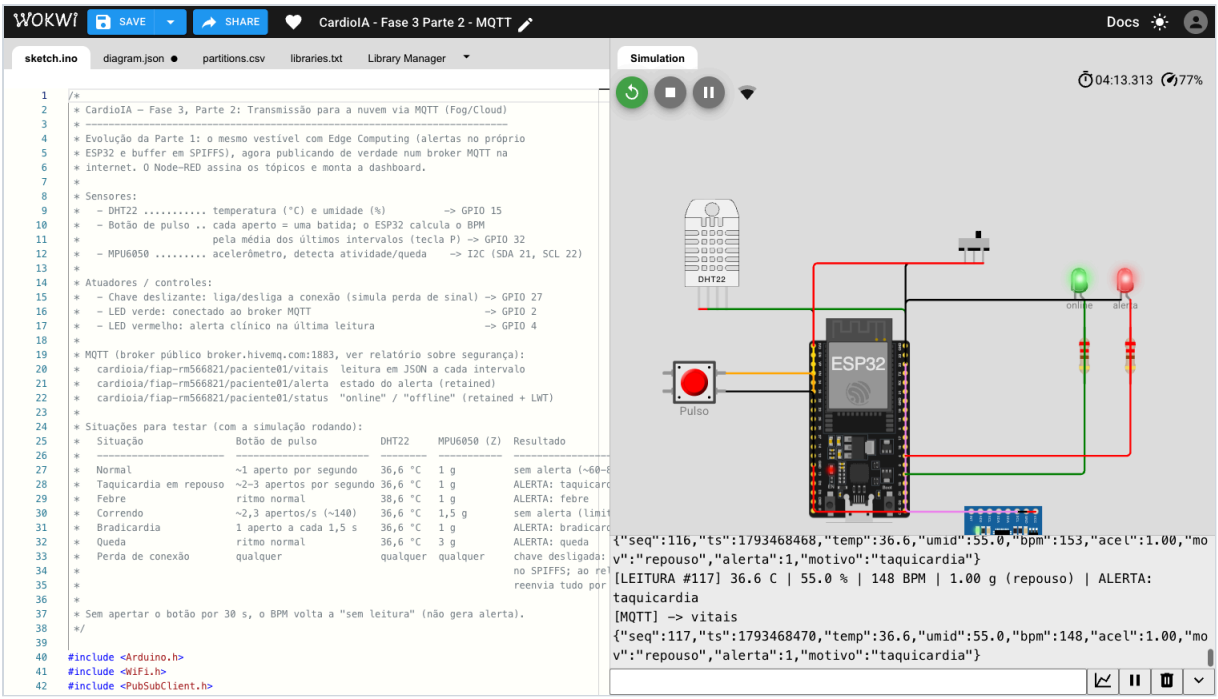
A Parte 1 deixou o vestível capaz de medir, decidir e guardar sozinho. Nesta parte ele passa a **falar com o mundo**: cada leitura sai do ESP32 por MQTT, atravessa um broker na internet e chega a uma dashboard no Node-RED, que mostra os sinais em tempo real e acende um alerta quando um limite é ultrapassado.

O sistema tem três camadas:

Camada	Onde roda	Papel no CardioIA
Edge	ESP32 (Wokwi)	lê os sensores, calcula o BPM, aplica as regras de alerta, guarda a fila no SPIFFS e publica por MQTT
Cloud	broker MQTT `broker.hivemq.com`	recebe as publicações e entrega a quem assinou os tópicos; guarda o último alerta e o status (retained)
Fog	Node-RED no computador da equipe de saúde	assina os tópicos, descarta repetidos, prepara os dados e serve a dashboard

O Node-RED fica na camada *fog*: está entre o dispositivo e o usuário, perto de quem consome os dados (o posto de enfermagem, por exemplo). Ele faz o pré-processamento antes da tela, como a deduplicação e a conversão de horário, sem depender de um serviço central.

O circuito é o da Parte 1, com uma troca: **o potenciômetro deu lugar a um botão de pulso**, como sugere o enunciado. Cada aperto é uma batida. O ESP32 mede o intervalo entre os apertos e calcula o BPM pela média dos últimos cinco intervalos (BPM = 60.000 ÷ intervalo médio em ms). Duas proteções deixam a medida confiável: um *debounce* de 200 ms descarta o repique mecânico do botão, e uma pausa maior que 2,5 s reinicia a medição. Sem essa segunda regra, o tempo parado entraria na média e geraria uma bradicardia falsa, defeito que encontramos nos testes e corrigimos. Sem apertos por 30 s, o BPM volta a "sem leitura", que não dispara alerta: falta de dado não é bradicardia.



Circuito da Parte 2 no Wokwi publicando por MQTT, com o alerta de taquicardia ativo

2. Fluxo de comunicação MQTT

MQTT é um protocolo de **publicação e assinatura**. O ESP32 não conhece a dashboard, e a dashboard não conhece o ESP32: os dois só conhecem o **broker** e os **tópicos**. Por isso o mesmo dado pode alimentar ao mesmo tempo a dashboard, um banco de dados e um aplicativo de celular, sem mudar nada no dispositivo.

Tópicos

Todos ficam sob o prefixo `cardioia/fiap-rm566821/paciente01/`. O broker é público, então o RM no caminho evita misturar mensagens com outros grupos.

Tópico	Quem publica	Conteúdo	Retained	Para quê
`.../vitais`	ESP32, a cada 2 s	leitura completa em JSON	não	gráficos, gauge e movimento
`.../alerta`	ESP32, quando o alerta muda	`{"ativo":true,"motivo":"taquicardia",...}`	**sim**	quem abre a dashboard depois já vê se o paciente está em alerta
`.../status`	ESP32 e o próprio broker	`online` ou `offline`	**sim**	indicador "Dispositivo ONLINE/OFFLINE"

Exemplo de mensagem em `vitais`:

```
{"seq":112,"ts":1793468440,"temp":36.6,"umid":55.0,"bpm":147,
 "acel":1.00,"mov":"repouso","alerta":1,"motivo":"taquicardia"}
```

`seq` é o número de série da leitura e `ts` é o horário real em segundos, vindo do NTP. O ESP32 acerta o relógio pela internet logo depois de entrar no Wi-Fi e só começa a medir depois disso, para nenhuma leitura sair datada de 1970.

Sequência de uma leitura

1. **Conexão.** O ESP32 entra na rede `Wokwi-GUEST`, acerta o relógio e abre uma sessão MQTT com o broker (porta 1883, *keep-alive* de 15 s, identificador aleatório `cardioia-xxxxxxxx`).
2. **Testamento (LWT).** Ao conectar, o ESP32 registra no broker um "último desejo": se ele sumir sem se despedir (bateria acabou, sinal caiu), o **próprio broker** publica `offline` em `status`. Logo depois o ESP32 publica `online`, também com *retain*.
3. **Medição e decisão no Edge.** A cada 2 s o ESP32 lê os sensores, calcula o BPM, classifica o movimento e aplica as regras de alerta (as mesmas da Parte 1).
4. **Fila e publicação.** A leitura vira JSON, entra na fila do SPIFFS e, se houver conexão, é publicada em `vitais` e sai da fila.
5. **Alerta.** Se o estado do alerta mudou (normal → alerta ou o contrário), o ESP32 publica em `alerta` com *retain*.
6. **Recepção no Fog.** O Node-RED, assinante dos três tópicos, recebe a mensagem em milissegundos e atualiza a dashboard de quem estiver com ela aberta.

Qualidade de serviço (QoS)

As leituras são publicadas com **QoS 0** ("no máximo uma vez"), o único nível de publicação da biblioteca PubSubClient. A entrega fica garantida pela aplicação: a leitura só sai da fila do SPIFFS depois que a publicação é aceita pela conexão. Se a conexão cair no meio, o que não saiu é regravado e reenviado depois. Em produção, com uma biblioteca que suporte QoS 1 ("pelo menos uma vez"), o broker confirmaria cada mensagem. Os repetidos que o QoS 1 pode gerar são descartados pelo Node-RED, como descrito abaixo.

Resiliência: o que acontece quando a conexão cai

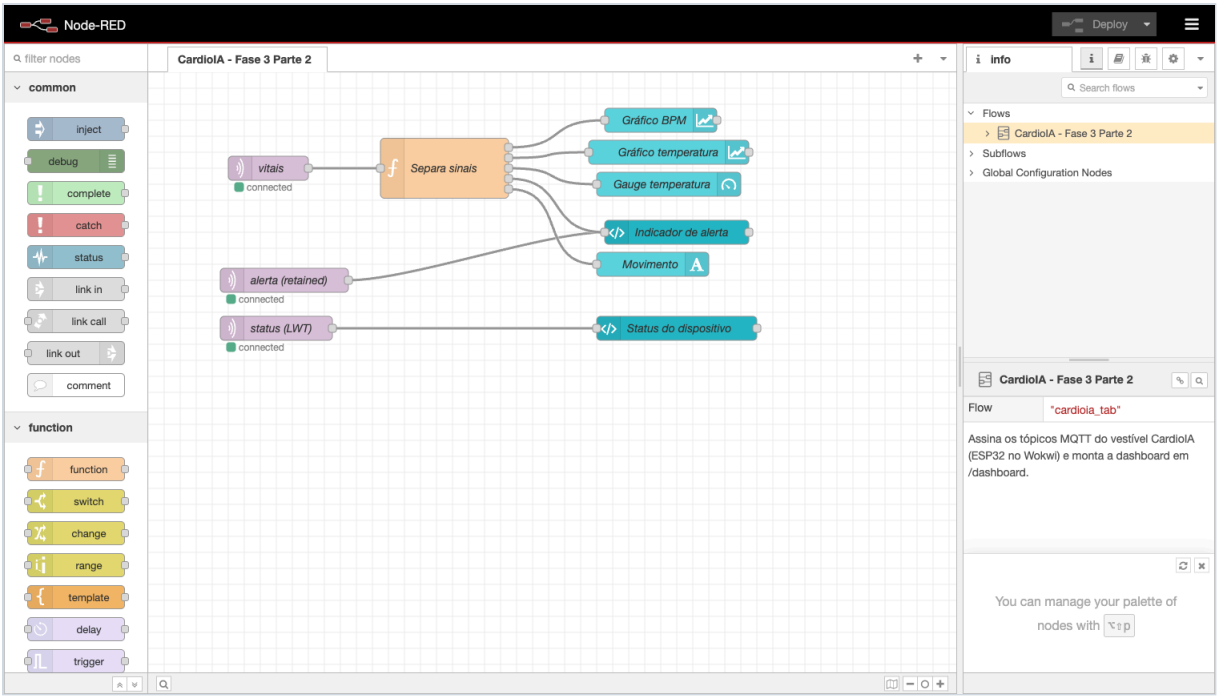
A chave deslizante simula o paciente sair da área de sinal.

- Ao desligar, o ESP32 publica `offline` e encerra a sessão; a dashboard mostra "Dispositivo OFFLINE" na hora.
- As leituras continuam sendo feitas e ficam na fila do SPIFFS, com a mesma compactação da Parte 1 (todas as leituras com alerta são preservadas).
- Ao religar, o ESP32 reconecta, publica `online`, republica o estado atual do alerta e **reenvia a fila em ordem**. Cada leitura carrega o horário em que foi medida, então o gráfico preenche o buraco no lugar certo da linha do tempo.
- No teste das capturas, uma taquicardia que começou **sem conexão** apareceu no gráfico depois da reconexão, nos instantes em que realmente aconteceu.
- O Node-RED guarda as últimas 200 chaves `ts-seq` e descarta repetidos, o que torna o reenvio seguro mesmo se o ESP32 reiniciar no meio de uma descarga.

3. Configuração da dashboard (Node-RED)

A dashboard usa o **Node-RED 4** com o **FlowFuse Dashboard 2.0** (`@flowfuse/node-red-dashboard`), sucessor do `node-red-dashboard` original, que foi descontinuado. O fluxo inteiro está em

`fase3/parte2/node-red/flows.json` e pode ser importado em qualquer Node-RED (menu → *Import*).

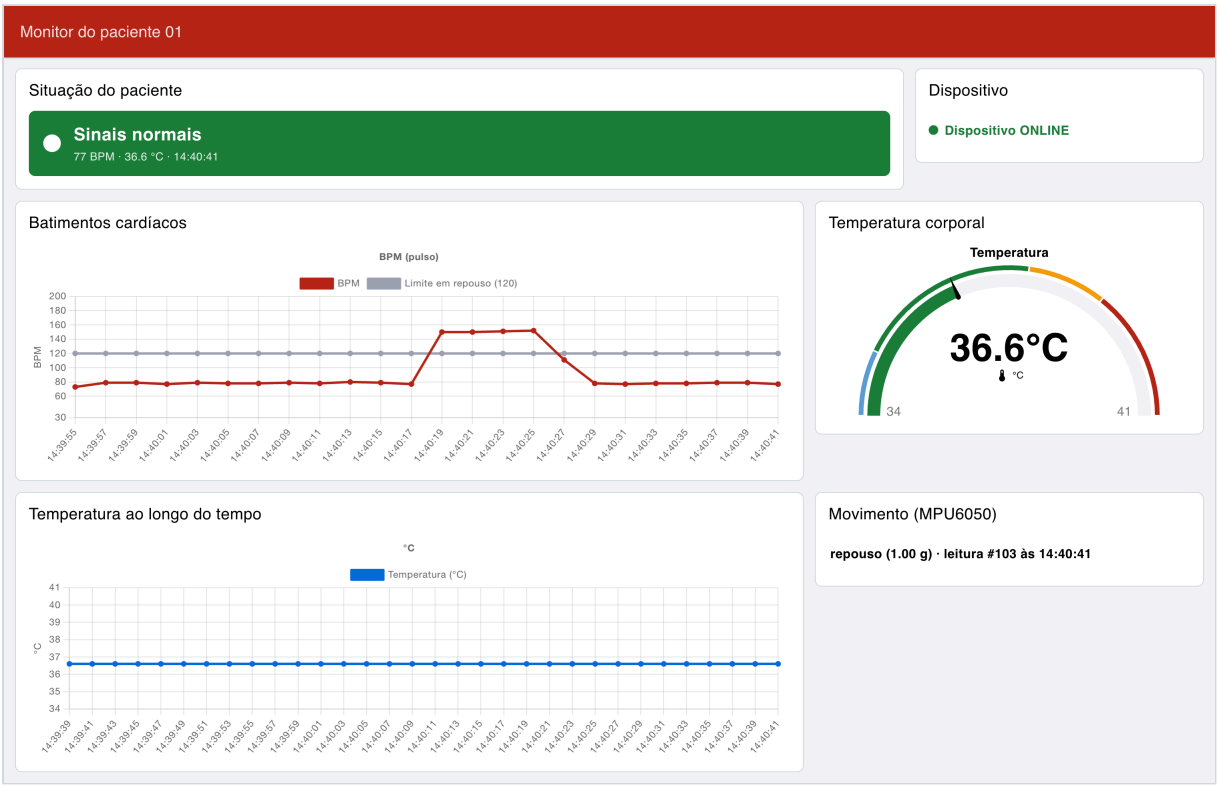


Fluxo no editor do Node-RED: três assinaturas MQTT, a função que separa os sinais e os widgets

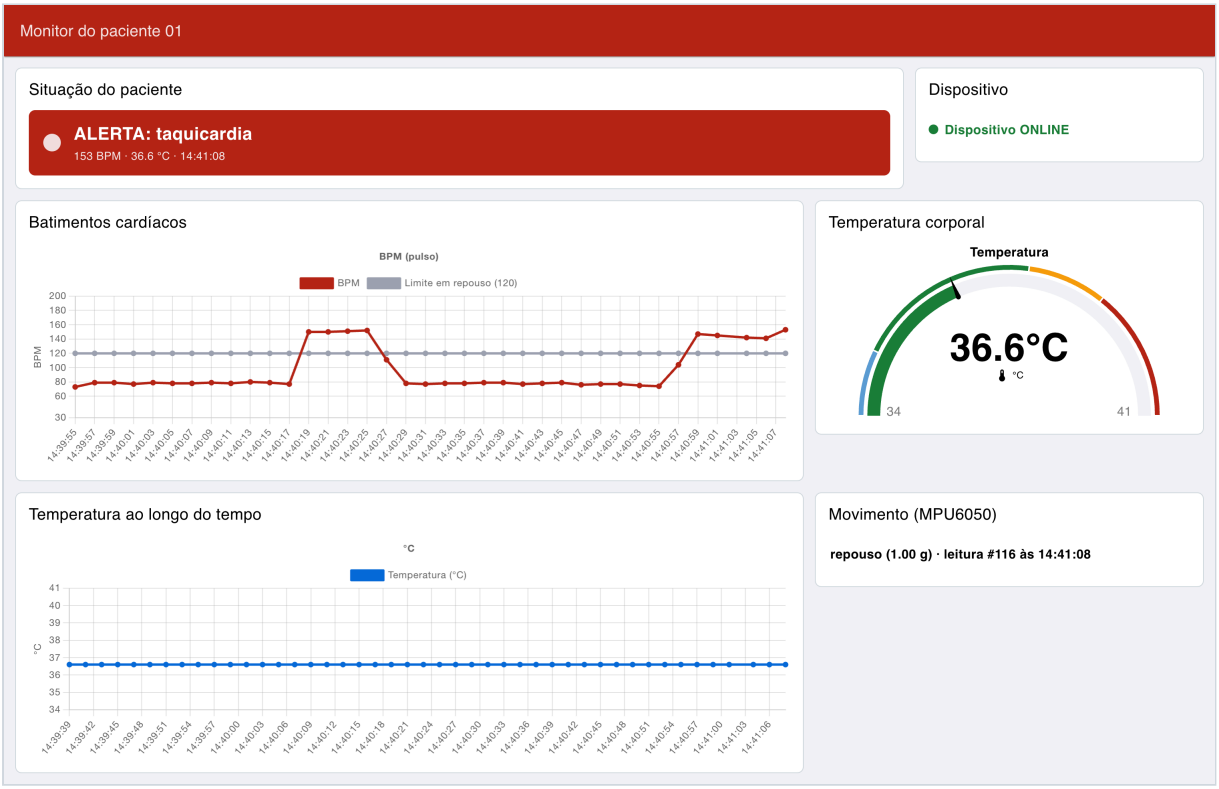
Nó	Tipo	Configuração
`vitais`	mqtt in	tópico `.../vitais`, saída como objeto JSON
`alerta (retained)`	mqtt in	tópico `.../alerta`, JSON; recebe o último alerta mesmo se a dashboard abrir depois
`status (LWT)`	mqtt in	tópico `.../status`, texto
`Separa sinais`	function, 5 saídas	descarta repetidos (`ts-seq`), converte o horário e monta uma mensagem para cada widget
`Gráfico BPM`	ui-chart (linha)	séries "BPM" e "Limite em repouso (120)", eixo Y de 30 a 200, janela de 5 min, eixo X em `HH:mm:ss`
`Gráfico temperatura`	ui-chart (linha)	°C de 34 a 41, mesma janela
`Gauge temperatura`	ui-gauge (meia-lua)	34 a 41 °C; faixas azul (< 35, hipotermia), verde (35–37,8), laranja (37,8–39, febre) e vermelha (≥ 39)
`Indicador de alerta`	ui-template (Vue)	faixa verde "Sinais normais" ou vermelha "ALERTA: motivo", com BPM, temperatura e hora; o "LED" pisca em alerta
`Status do dispositivo`	ui-template	"Dispositivo ONLINE" (verde) ou "OFFLINE" (cinza), vindo do LWT
`Movimento`	ui-text	repouso, atividade ou queda, com a aceleração e o número da leitura

Limites definidos pelo grupo. O alerta segue as regras clínicas do Edge: taquicardia acima de **120 BPM em repouso** (150 BPM em atividade), bradicardia abaixo de 50 BPM, febre a partir de **37,8 °C**, hipotermia até 35 °C e queda acima de 2,5 g. A dashboard **não recalcula** o alerta: ela mostra a decisão tomada no dispositivo. Assim o alerta funciona até sem a dashboard (o LED vermelho do vestível acende do mesmo jeito), e os dois lados nunca discordam. A linha cinza do gráfico mostra o limite de 120 BPM para o profissional enxergar o quanto o valor passou.

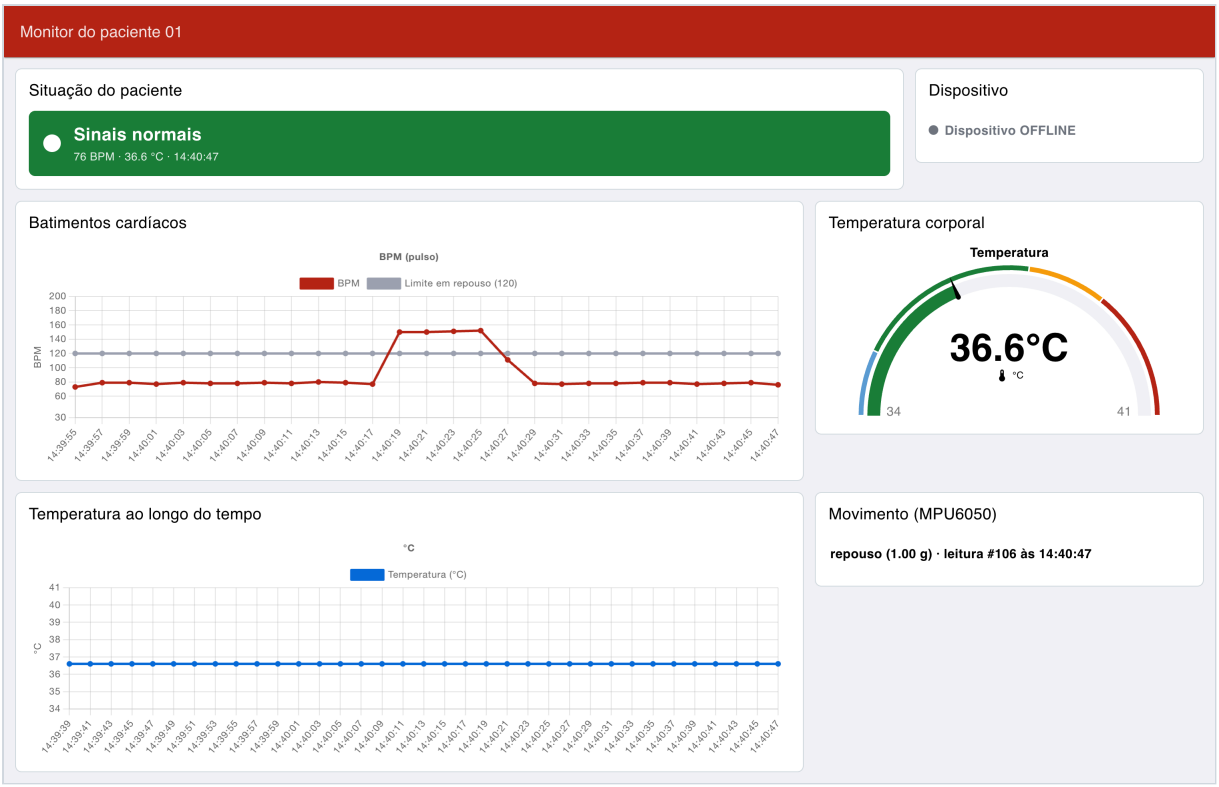
Layout. Uma página, "Monitor do paciente 01", numa grade de 12 colunas. Na primeira linha, a situação do paciente (o mais importante, no topo) e o status do dispositivo. Na segunda, o gráfico de batimentos e o gauge de temperatura. Na terceira, a temperatura ao longo do tempo e o movimento.



Dashboard em situação normal: taquicardia passada visível no gráfico, alerta verde



Dashboard em alerta: faixa vermelha com o motivo, BPM acima da linha de 120



Dashboard com o dispositivo offline: status cinza enquanto as leituras ficam na fila do ESP32

4. Segurança e limitações

Broker público. O enunciado sugere o HiveMQ Cloud, mas o plano gratuito (Serverless) foi descontinuado: hoje só existe o plano Starter, pago, com teste de 15 dias. Usamos o broker público `broker.hivemq.com`, também citado como exemplo. Ele funciona para demonstrar o fluxo, mas **não tem autenticação nem criptografia**: qualquer pessoa que souber o tópico pode ler ou publicar. Por isso o payload não traz nenhum dado que identifique o paciente, só `paciente01`.

Em produção a mesma arquitetura precisaria de:

- TLS na porta 8883 (`WiFiClientSecure` no ESP32 e certificado do broker);
- usuário e senha por dispositivo, ou certificado de cliente;
- ACL no broker: cada vestível só publica nos próprios tópicos, e só a central lê `cardioia/#`;
- QoS 1 nas leituras e persistência da sessão;
- tratamento de dado de saúde como dado sensível (LGPD, art. 11): base legal, consentimento e registro de acesso.

Simulação. O pulso vem de um botão, não de um sensor óptico, e a perda de conexão vem de uma chave. O Node-RED roda localmente; num hospital ele estaria num servidor da rede interna ou num serviço gerenciado.

5. Como reproduzir

1. Abrir o projeto no Wokwi (link no topo) e iniciar a simulação. A chave deslizante liga a conexão; o LED verde acende quando o MQTT conecta.
2. Na pasta `fase3/parte2/node-red`: `npm install` e depois `npm start`.
3. Abrir `http://127.0.0.1:1880/dashboard`.
4. Apertar o botão de pulso (ou a tecla **P** com o mouse sobre o simulador) uma vez por segundo para ver ~60 BPM, e mais rápido para disparar a taquicardia.
5. Desligar a chave, continuar apertando e religar para ver o reenvio da fila.